G091 / 이병주

AI 서비스 Mini-Project / 과제 1

RAG와 선택적 멀티에이전트를 활용한 운영 장애 대응 에이전트 개선

1. 진행 사항 소개

기존 Base AI Agent의 조사·검색·검증 흐름을 개선해, 확보한 자료에 맞는 장애 대응 답변을 생성하도록 구성했다.

> **출발점: 확인하지 않은 사실을 단정하는 답변**
> CPU가 높다는 한 번의 관측만으로 지속 고부하를 단정하고, 조회하지 않은 로그를 분석했다고 설명했다. 미등록 서비스를 다른 서비스로 대신 조사하는 문제도 있었다.

추가·수정한 기능

1. **대상·근거 확인:** 미등록 대상의 임의 대체를 막고 로그·시계열·배포 자료의 제공 여부를 구분했다.
2. **RAG:** 키워드 기반 단일 문서 선택을 벡터 유사도 기반 다중 Runbook 검색으로 확장했다.
3. **선택적 멀티에이전트:** 지표·로그·Runbook 분석을 분리하고 필요한 역할만 병렬 호출했다.
4. **검증·평가:** 의미 검증과 Fallback을 추가하고, 고정 평가 자료 및 동일 기준 재채점을 구성했다.

목차

1. 진행 사항 소개	1-2쪽
2. 사용한 기술 요소	3-7쪽
4. 개선 전/후 성능 비교	8-9쪽
5. 이후 반영하고 싶은 개선 포인트	10쪽

운영 지표는 실습용 모의 데이터이며 개선 대상은 에이전트의 답변 품질과 처리 구조다. 실제 운영 서비스의 CPU 개선이나 모델 재학습을 수행한 프로젝트는 아니다.

1. 진행 사항 소개 · 구현 결과

기존 회귀 사례에서 대상 선택 오류와 문서 검색 누락을 줄였다. 반면 토큰 사용량과 긴 응답 구간의 지연은 증가했으며, 새 질문에 대한 일반화 성능은 아직 검증하지 않았다.

대상 선택 정확성	Runbook 검색 재현율	총 토큰
90.9 → 100%	**77.8 → 100%**	**2.61배**
30/33 → 33/33 응답	정밀도 100 → 81.8%	29,547 → 77,093

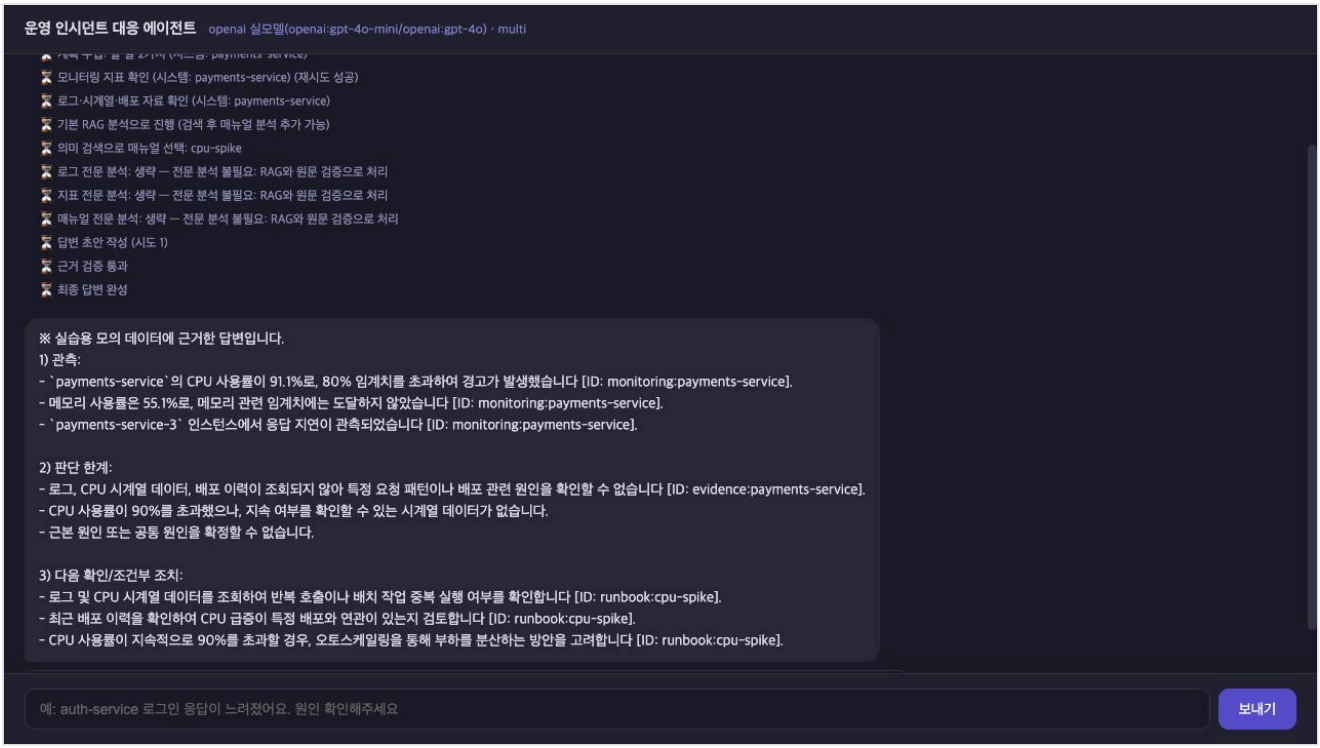


그림 1. CPU 고부하 질문의 최종 응답: 관측, 판단 한계, 다음 확인 및 조건부 조치를 구분한다.

질문 1에서 확인한 답변 변화

초기는 “최근 로그를 분석한 결과”와 “CPU 사용률이 90%를 넘고 지속되고 있으므로”라고 단정했다. 현재는 로그·시계열 자료 부족을 명시하고, 지속 고부하 확인 뒤 증설하도록 조건을 제시한다.

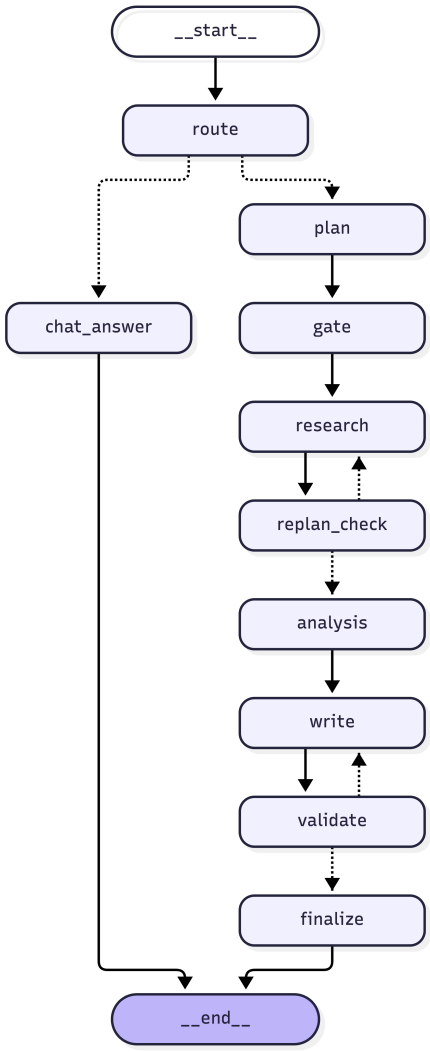
> **결과의 범위:** 10개 평가 사례를 각 3회 실행했다. 10번은 2턴 대화이므로 비교 대상은 버전별 응답 33건이다. 동일 기준 LLM 재채점에서 현재 점수가 높았으나 오판이 확인돼 100%를 검증된 정확도로 해석하지 않았다.

응답시간 중앙값 5.1390 → 5.2379초, p95 6.1630 → 8.4307초. 대상 처리는 코드 검사, 검색은 문서 ID 대조, 답변 품질은 동일 LLM 채점과 원문 검토로 확인했다.

2. 사용한 기술 요소 · Graph 구조

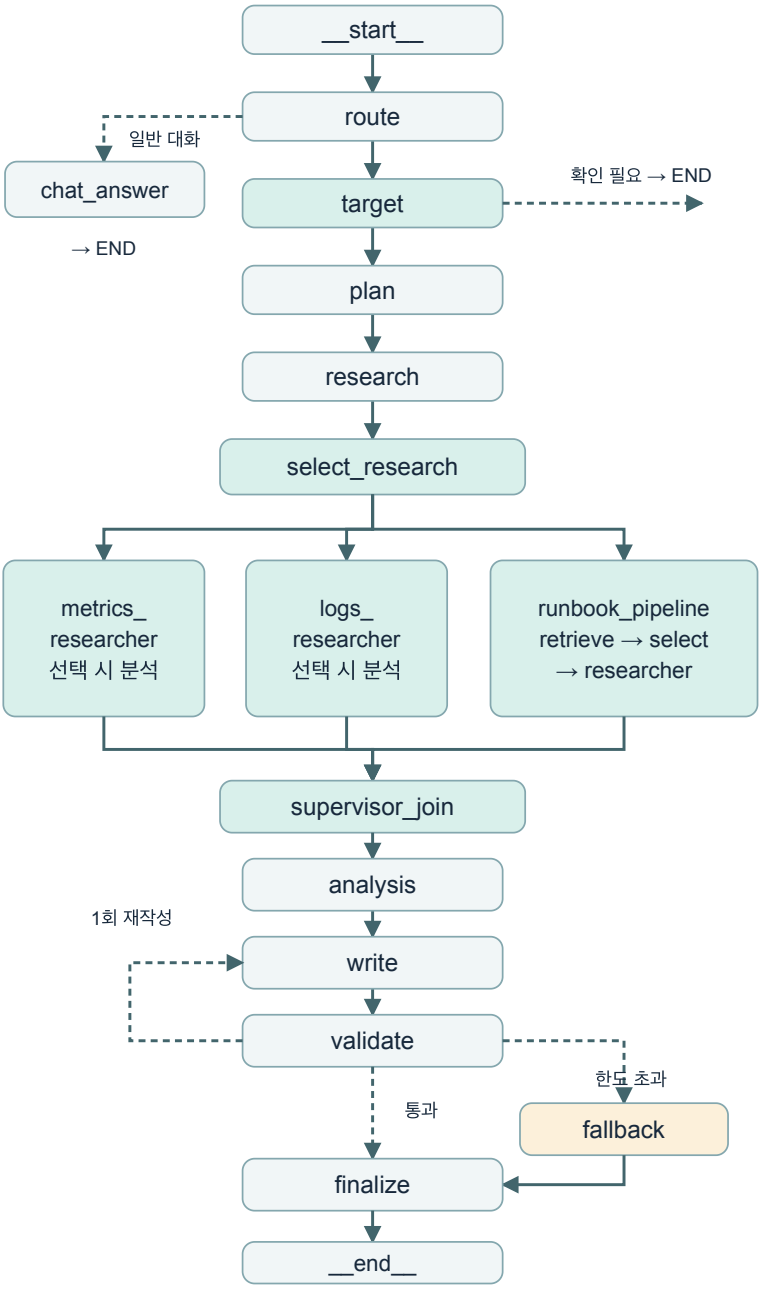
초기의 순차 조사·재계획 흐름을 **대상 확인 + RAG + 선택적 전문 분석 + 제한된 재검증** 흐름으로 변경했다. 아래 개선 구조는 현재 `multi` 모드 기준이다.

초기 구조



제공된 초기 구성도 원본
research ↔ replan_check

개선 구조



초록: 추가·분리한 처리 / 노랑: 실패 종료 경로
전문가가 선택되지 않으면 해당 LLM 호출을 생략한다.

실선은 고정 연결, 점선은 조건 분기다. Runbook 서브그래프는 검색→역할 선택→전문 분석으로 구성된다. 일반 대화는 chat_answer에서 종료한다.

2. 사용한 기술 요소 · Agent Graph

기존 `build_graph()`를 유지하면서 Research를 분리했다. **노드 추가뿐 아니라 분기·합류·실패 종료 경로를 변경**해 조사 범위와 재작성 횟수를 제어한다.

① 전문가 선택 이후 세 분기로 실행하고 합류

`agent/graph.py · 48–60행`

```
# 추가: 지표·로그와 Runbook 검색/분석을 독립 분기로 연결하고 모두 완료되면 합류
graph.add_node('select_research', select_research_node)
graph.add_edge('research','select_research')
workers = []
for role in ('metrics','logs'):
    name = f'{role}_researcher'
    graph.add_node(name,worker_node(role,deps.fast_llm))
    graph.add_edge('select_research',name)
    workers.append(name)
graph.add_node('runbook_pipeline', runbook_branch(deps))
graph.add_edge('select_research','runbook_pipeline')
workers.append('runbook_pipeline')
graph.add_node('supervisor_join',supervisor_join)
graph.add_edge(workers,'supervisor_join')
```

`workers` 목록을 출발점으로 지정한 Edge가 세 분기의 완료를 기다린다. Runbook 검색은 지표·로그 분석의 완료를 기다리지 않고 진행한다.

② 노드를 지나더라도 불필요한 모델 호출은 생략

`agent/researchers.py · 89–96행`

```
# 개선: 역할별 enabled를 검사해 불필요한 전문 LLM 호출과 토큰 사용을 줄임
selected = state.get("research_selection", {}).get(role, {"enabled": True})
# [선택적 호출] 그래프 노드는 지나가더라도 선택되지 않은 역할은 모델을 호출하지 않는다.
if not selected["enabled"]:
    return {"findings": [{"role": role, "status": "skipped",
        "reason": "전문 분석 불필요: RAG와 원문 검증으로 처리", "seconds": 0.0}]}
result = await subgraph.ainvoke({"query":state["query"],
                                 "evidence":select_evidence(state, role)})
return {"findings": [result["finding"]]}
```

③ 의미 검증 실패는 한 번 재작성한 뒤 Fallback

`agent/graph.py · 65–67행`

```
# 변경: 검증 상태별 조건부 Edge로 재작성·정상 종료·제한 답변을 구분
graph.add_conditional_edges('validate',lambda s:
{'passed':'finalize','retry':'write','exhausted':'fallback'}[s['validation_status']],
{'finalize':'finalize','write':'write','fallback':'fallback'})
graph.add_edge('fallback','finalize')
graph.add_edge('finalize',END)
```

추가한 target은 미등록·불명확 대상을 확인 안내로 종료한다. 기존 gate는 제거했으며, 현재 multi 모드는 replan_check 순환 대신 RAG 분기를 사용한다. 작성·검증 노드에는 원문 근거를 함께 전달한다.

2. 사용한 기술 요소 · MCP

기존 MCP 서버와 모니터링 Tool을 활용하고, **추가 진단 근거 Tool**을 만들었다. 핵심은 자료가 없을 때 이를 “정상”이나 “특이 패턴 없음”으로 바꾸지 않는 것이다.

① 로그·CPU 시계열·배포 자료의 제공 여부 명시

mcp_server.py · 69–83행

```
# 추가: 근거 종류별 availability와 records를 반환해 미제공과 정상 관측을 구분
@mcp.tool()
def get_diagnostic_evidence(system_name: str) -> dict:
    """명시적 모의 진단 자료 조회. 자료 미제공은 정상 또는 이상 없음이 아니다."""
    registry = json.loads((DATA_DIR / "monitoring.json").read_text())
    if system_name not in registry:
        raise ValueError("System not registered")
    path = DATA_DIR / "diagnostic_evidence.json"
    data = json.loads(path.read_text()) if path.exists() else {}
    source = data.get(system_name, {})
    return {
        "system_name": system_name,
        "data_kind": "synthetic",
        **{kind: source.get(kind, {"availability": "not_provided", "records": []})
           for kind in ("logs", "cpu_series", "deployments")},
    }
```

> `not_provided`는 자료가 없다는 뜻이다. 로그 내용이나 지속 시간을 추정할 근거가 아니며, 작성 단계에는 확인할 수 없는 항목으로 전달한다.

② 에이전트에서 작업 종류에 맞는 Tool 호출

agent/nodes/research.py · 33–35행

```
# 변경: 모니터링 작업과 추가 근거 작업을 구분하고 조회 결과에 근거 ID를 부여
    tool='get_monitoring_status' if task['kind']=='monitoring' else 'get_diagnostic_evidence'
    data,retries=await with_retry(deps.mcp.call_tool,tool,{'system_name':task['target']})
return {'id':f"{task['kind']}:
{task['target']}",'source':task['kind'],'target':task['target'],'data':data,'retries':retries}
```

대상별로 monitoring·evidence 작업을 계획하고, MCP 클라이언트를 통해 조회한다. 결과에는 자료 종류, 대상, 재시도 횟수가 함께 저장되어 작성·검증에 사용된다.

기존 기능과 추가 기능의 구분

기존 `get_monitoring_status`와 Runbook Resource·Prompt는 보존했다. 추가한 Tool은 진단 자료를 수집하며, 현재 multi 경로의 Runbook 검색은 별도 벡터 검색기에서 수행한다. 계획은 코드로 구성한다.

현재 자료 원천은 로컬 모의 JSON이다. 작업 예약은 병렬이지만 MCP 클라이언트의 실제 호출은 잠금으로 직렬화된다. 전문 분석·RAG 분기의 병렬 실행과 구분해야 한다.

2. 사용한 기술 요소 · RAG와 LCEL

Runbook 파일을 늘리는 대신 **기존 문서를 선택하는 방식**을 바꿨다. 본문을 임베딩하고, 정규화한 벡터의 내적으로 코사인 유사도를 계산한다.

① 질문·관측 알림을 각각 검색어로 구성

agent/researchers.py · 147–150행

```
# 추가: 복합 질문에서 일부 증상이 묻히지 않도록 질문과 서비스별 알림을 분리 검색
metrics = [c["data"] for c in state["collected"] if c["source"] == "monitoring" and not
c.get("error")]
# Separate observation queries keep a composite question from drowning out one symptom.
queries = [state["query"]] + [
    "관측 알림: " + "; ".join(m.get("alerts", [])) for m in metrics if m.get("alerts")]
```

② 유사도·순위 제한 후 검색어별 결과를 중복 제거

agent/retrieval.py · 116–120행

```
# 추가: 유사도 기준 미달 문서는 제외하고 검색어당 상위 후보를 제한
def select(self, ranked):
    if not ranked or ranked[0]["score"] < self.threshold:
        return []
    cutoff = max(self.threshold, ranked[0]["score"]-self.margin)
    return [d for d in ranked[:self.top_k] if d["score"] >= cutoff]
```

agent/retrieval.py · 124–131행

```
# 추가: 검색어별 후보를 합쳐 문서 ID로 중복 제거하고 다중 Runbook을 반환
async def search(self, queries):
    rankings = await self.rank(queries)
    unique = {}
    for ranking in rankings:
        for doc in self.select(ranking):
            if doc["id"] not in unique or doc["score"] > unique[doc["id"]]["score"]:
                unique[doc["id"]] = doc
    return sorted(unique.values(), key=lambda d: (-d["score"],d["id"]))
```

현재 설정은 검색어당 top_k=1, threshold=0.25, margin=0.12다. 검색어가 여러 개이므로 최종 문서는 여러 개일 수 있다. SQLite에 문서·벡터를 저장하고 문서 해시나 모델이 바뀐 문서만 다시 임베딩한다.

③ LCEL로 모델 호출 구조 통일

agent/chains.py · 47–48행

```
# 모듈화: 프롬프트 → 모델 어댑터 → 파서를 체인으로 연결하고 사용량도 함께 유지
return (prompt | RunnableLambda(call_model, name=f"{name}.model")
        | RunnableLambda(parse_output, name=f"{name}.parse")).with_config(run_name=name)
```

검색 결과는 근거 ID와 함께 수집 상태에 추가한다. 복수 후보의 적용성 판단이 필요하면 Runbook 전문가가 분석한다. 현재는 재현율이 높아진 대신 불필요한 문서도 유입되므로, 검색 성공과 조치의 타당성을 별도로 검증해야 한다.

2. 사용한 기술 요소 · 적용 범위

과제의 8개 개선 항목	적용 내용
1. RAG	임베딩·유사도 기반 검색, 다중 Runbook 결합
2. 모듈화	그래프·노드·정책·검색·전문 분석·출력 분리
3. Research 멀티에이전트화	지표·로그·Runbook 서브그래프와 선택적 병렬 실행
4. Human-in-the-loop	미적용. 자동 Reviewer와 사람 승인은 구분
5. 대화 Context 요약	미적용. 전문 분석 출력 축소는 장기 대화 요약과 별개
6. LLM-as-judge 평가 확장	고정 질문별 기준, 동일 기준 재채점, 원문 검토
7. 하드코딩 개선	부분 적용. 기본 서비스 임의 대체 방지 및 대상 확인
8. 데이터 보강	로그·시계열·배포 자료 구조와 경계 사례 5개 추가

추가 개선사항

LCEL 호출 구조, 의미 검증과 Fallback, 선택적 전문가 호출, 전문 분석 길이 제한, 사용량 계측을 추가했다. 의존성 주입으로 실제 모델·MCP·검색기를 평가용 구현으로 교체하고, LangSmith 추적과 단위·회귀 테스트를 활용했다.

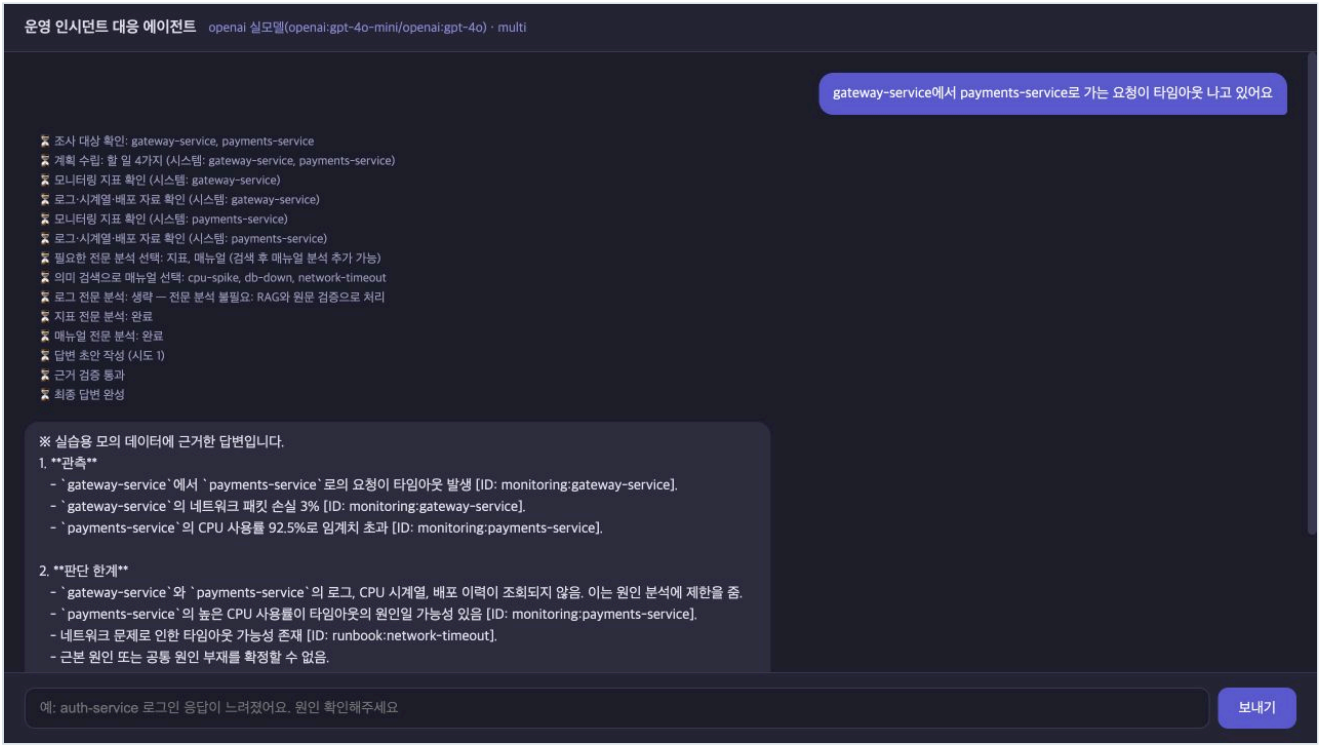


그림 2. 복합 질문에서 두 서비스를 조사하고 지표·Runbook 분석을 수행한다. 자료가 없는 로그 분석은 생략한다.

검증 기록: 단위·회귀 테스트 44개, 고정 평가 사례 15개의 데이터 검사 통과. 기본 사례 10개와 추가 경계 사례 5개는 구분해 평가했다.

4. 개선 전/후 성능 비교

최초 원본과 현재의 대응하는 응답을 각각 33건씩 비교했다. 10개 평가 사례를 각 3회 실행했으며, q10은 2턴으로 총 응답 수가 33건이다. 현재에만 있는 추가 경계 사례 15건은 이 비교에서 제외했다.

지표	초기	현재	차이·해석
응답시간 중앙값	5.1390초	5.2379초	+0.0989초 (+1.9%)
응답시간 p95	6.1630초	8.4307초	+36.8%
등록 장애 21건 중앙값	5.4410초	5.7226초	+5.2%
총 토큰	29,547	77,093	2.61배 (+160.9%)
대화 / 질문 임베딩 토큰	29,547 / 0	75,833 / 1,260	현재 총량에 임베딩 포함
대화 모델 호출	99	78	-21회 (-21.2%)
답변 재작성 / 도구 retry	0 / 0	0 / 0	변화 없음

비용: 기록 종류를 구분해 비교

구분	초기	현재
기존 앱의 가상 비용 지표 state.cost_usd / 청구액 아님	$0.072952	$0.308321 기록상 4.23배
입력·출력·캐시별 비용 추정	초기 계측에서 입력·출력·캐시 토큰 미저장	$0.201955 질문 임베딩 포함, 평가자 비용 제외

초기 99회 모델 호출은 총 토큰과 가상 비용만 저장했다. 입력·출력·캐시별 사용량은 현재 버전부터 기록했으므로, 보존된 초기 기록만으로 동일 방식의 비용을 복원할 수 없다. 가상 비용과 입력·출력별 추정 비용을 교차 비교할 수 없으며, 현재 추정 비용에는 문서 최초 임베딩 비용이 포함되지 않는다.

측정 조건과 해석

초기 2026.10.20, 현재 2026.10.21 실행 기록이다. 질문·반복·턴, 데이터 매니페스트와 Runbook 해시가 일치하고, 공통 조회 대상의 모니터링 값도 동일했다. 모델 구성은 gpt-4o-mini/gpt-4o, temperature=0이며 무작위 지연·도구 오류 주입이 없는 고정 MCP를 사용했다. p95는 nearest-rank 방식이다.

> 전체 중앙값은 비슷하지만 긴 응답 구간과 총 토큰은 증가했다. 실행 시점·API 응답 편차가 있으므로 속도 차이는 보존 기록의 관측 결과로 해석한다. 모의 CPU 수치의 하락을 측정한 결과가 아니다.

출처: `eval/results/before-live/responses.jsonl` · `eval/results/current/responses.jsonl`
재집계: `output/pdf/report-comparison.json`

4. 개선 전/후 성능 비교 · 품질

동일 기준으로 저장 응답 66건 재채점

초기·현재 각각 33건을 같은 gpt-4o, temperature=0, 평가 지침으로 채점했다. 버전명을 숨기고 순서를 섞었으며, 질문별 기준·Runbook·실제 도구 결과와 답변을 대조했다. 일반 대화와 미등록 대상은 조치 조건·다음 행동 평가에서 제외했다.

자동 채점 통과율	초기	현재
근거 충실성	27/33 (81.8%)	33/33 (100.0%)
조치 조건 준수	18/21 (85.7%)	21/21 (100.0%)
요청 대응	27/33 (81.8%)	33/33 (100.0%)
구체적 다음 행동 제시	17/21 (81.0%)	21/21 (100.0%)
무관한 조치 포함 ↓	3/33 (9.1%)	0/33 (0.0%)

> **자동 점수의 한계:** 평가자는 초기의 미조회 로그 분석·지속 고부하 단정뿐 아니라 현재의 '90% 초과→이상' 변경과 지속 조건 누락도 통과시켰다. 이 수치는 동일 평가자의 자동 통과율이며 검증된 최종 정확도가 아니다.

검색 결과는 같은 문서 ID 기준으로 대조

기본 장애 사례 21개 응답	초기	현재
정밀도 / 재현율	100% / 77.8%	81.8% / 100%
F1 / 문서 집합 완전 일치율	87.5% / 71.4%	90.0% / 71.4%

q04·q06에서 필요한 문서의 누락을 줄였지만 q04의 db-down, q05의 cpu-spike가 추가 유입됐다. 별도 검색 평가 질문 12개에서는 RAG F1이 57.1%였다.

결론

기존 회귀 사례에서 대상 처리와 일부 근거 표현은 개선됐다. 다만 개발에 사용한 질문의 반복 평가이며, 자동 평가에도 오판이 남아 있다. 새로운 질문에 대한 일반화나 전체 성능 향상을 확정하지 않는다.

재채점: `eval/results/rejudged-20261021/summary.json · rubric.txt`
원문 검토: `eval/results/rejudged-20261021/audit_findings.json`
검색 재집계: `output/pdf/report-comparison.json`

5. 이후 반영하고 싶은 개선 포인트

① 개발용 질문과 최종 평가용 질문 분리

같은 10개 사례를 보고 프롬프트·검색·분기를 수정했으므로 과적합 위험이 있다. 현재 코드를 고정한 뒤, 사용하지 않은 서비스 조합·임계값·자료 누락·충돌 사례로 초기와 현재를 비교해야 한다. 질문 표현만 바꾸는 평가로는 충분하지 않다.

② LLM 평가와 결정적 검사를 함께 사용

동일 모델과 지침을 사용해도 평가의 정확성이 보장되지는 않았다. 수치 비교·대상 일치·근거 ID는 코드로 검사하고, 인과관계·자료 부재·조치 전제는 원문과 대조해야 한다. 자동 점수와 검토 결과는 구분해 보존한다.

③ 검색 정밀도와 비용·지연의 균형

다중 문서 검색은 누락을 줄였지만 불필요한 문서를 포함했다. 선택적 전문가 호출과 출력 축소를 적용했어도 최초 버전 대비 토큰은 2.61배, p95 지연은 36.8% 증가했다. 관련성 필터와 입력 축소는 새로운 평가 사례에서 품질 저하 여부와 함께 검증해야 한다.

④ 초기·현재 비용 계측 통일

초기 기록에는 입력·출력·캐시 토큰이 없어 현재의 비용 추정 방식과 직접 비교할 수 없다. 최초 그래프 동작은 보존하고 계측만 보강해 다시 측정하되, 기존 Before 기록과 새 실행 결과를 구분해야 한다.

⑤ 실제 조치·장기 대화의 후속 설계

현재는 조치 권고 중심이며 사람 승인과 장기 대화 요약은 미구현이다. 실제 재시작·증설에 연결할 경우 권한·승인·실행 결과 확인을, 장기 대화에는 요약 시 정보 손실과 비용 감소를 함께 설계해야 한다.

주요 근거 자료

- `agent/graph.py` · `agent/researchers.py` · `agent/chains.py` — 그래프·전문 분석·LCEL
- `mcp_server.py` · `agent/nodes/research.py` · `agent/retrieval.py` — 근거 수집·벡터 검색
- `eval/baseline/` · `eval/results/before-live/` · `eval/results/current/` — 최초 코드와 실행 결과
- `eval/results/rejudged-20261021/` — 동일 기준 재채점·원문 검토
- `eval/results/retrieval/results.json` · `output/pdf/report-comparison.json` — 검색·성능 집계